

Software Requirements Specification (SRS)

Niche Multivendor Ecommerce Marketplace

Developer-Ready System Specification with AI Agent Build Guide

Prepared for: Solo Developer Building with AI Agent Support

Document Version: 2.0

Date: 2026-05-10

Core Principle

Build a controlled, trust-first marketplace operating system - not just a simple ecommerce website.

Document Control

Item	Value
Document name	Software Requirements Specification with AI Agent Build Guide
System type	Niche multivendor ecommerce marketplace
Target builder	Solo developer using AI coding agents
Primary deliverable	Buyer storefront, seller dashboard, admin dashboard, marketplace operations engine
Version	2.0
Status	Developer-ready baseline specification

How to Use This SRS

This SRS is designed to be used directly during development. Keep it inside the project repository under /docs/SRS.pdf and use it as the master source of truth for product features, business rules, and AI-agent tasks.

- Use the requirement IDs when creating tasks, commits, pull requests, and AI prompts.
- Do not build features that conflict with the strict business rules in this document.
- For each module, complete the database model, API contract, UI, validation, tests, and admin controls before marking it done.
- Use the AI Agent Build Guide section to divide the project into small, reviewable tasks.
- Never let an AI agent rewrite large parts of the codebase without a clear task, acceptance criteria, and test plan.

Table of Contents

1. Introduction and Scope
2. Product Vision and Business Goals
3. User Roles and Permissions
4. System Modules
5. Functional Requirements
6. Strict Marketplace Business Rules
7. AI Agent Build Guide for Solo Developer
8. Recommended Technical Architecture
9. Data Model and Main Entities
10. API Requirements
11. Non-Functional Requirements
12. Security and Fraud Requirements
13. Testing and Quality Assurance
14. Deployment and DevOps
15. MVP Roadmap and Step-by-Step Build Plan
16. Final Launch Checklist

1. Introduction and Scope

1.1 Purpose

The purpose of this Software Requirements Specification is to define a complete, strict, and implementation-ready plan for building a niche multivendor ecommerce marketplace. The platform shall allow verified sellers to list and sell products, customers to purchase safely, and administrators to control trust, quality, payments, delivery, returns, commissions, and settlements.

This document is especially written for a solo developer working with AI coding agents. It therefore includes strict implementation rules, modular build phases, requirement IDs, acceptance criteria, AI prompt patterns, and quality checkpoints.

1.2 Project Assumption

The system is assumed to be a niche marketplace for local sellers and customers. The niche may be fashion, beauty, electronics accessories, home products, books, handmade products, or similar categories. The requirements are general enough to support multiple niches but strict enough to prevent weak marketplace architecture.

1.3 In Scope

- Customer storefront for browsing, searching, cart, checkout, order tracking, reviews, and returns.
- Seller dashboard for onboarding, product listing, inventory, order processing, returns, reports, and payouts.
- Admin dashboard for seller approval, product moderation, order control, finance, returns, support, marketing, and reports.
- COD and digital payment workflows.
- Commission calculation and seller settlement management.
- Delivery, shipment tracking, cancellation, return, exchange, and refund workflows.
- Coupons, campaigns, promoted listings, and content management.
- Security, audit logs, role-based access control, fraud controls, backups, and monitoring.

1.4 Out of Scope for MVP

- Full warehouse management system.
- Seller financing or loan module.
- Advanced AI recommendations.
- Native mobile apps, unless budget and timeline allow.
- Complex ERP integration.
- Live commerce and affiliate marketplace features.
- Full private-label supply chain management.

1.5 Definitions

Term	Definition
Marketplace	A platform where multiple sellers sell products to customers.
Seller or Vendor	A business or individual approved to sell on the platform.
Customer or Buyer	A user who browses and purchases products.
Admin	Platform operator who manages business and system operations.
SKU	Stock Keeping Unit used to identify product or variant inventory.
COD	Cash on Delivery.
Commission	Platform fee charged on seller sales.
Settlement	Seller payout after commission, refunds, fees, and adjustments.
RTO	Return to Origin due to failed delivery.
KYC	Identity and business verification for sellers.

2. Product Vision and Business Goals

2.1 Product Vision

The product vision is to build a trusted niche marketplace where customers can safely buy quality products from verified local sellers and sellers can grow using professional ecommerce tools.

The system must be designed as a marketplace operating system. This means it must handle more than product pages and checkout. It must also handle seller approval, product moderation, multi-seller order splitting, commission calculation, payout rules, customer disputes, returns, refunds, fraud signals, and marketplace analytics.

2.2 Business Goals

- Build trust through seller verification, product approval, reviews, transparent delivery, and clear return policies.
- Launch a focused MVP that can process real orders safely.
- Scale product catalog without owning all inventory.
- Generate revenue from commissions, promoted listings, seller services, and campaigns.
- Protect profitability by monitoring delivery cost, return rate, cancellation rate, COD failure, and seller performance.
- Enable a solo developer to build in stages using AI agents without losing control of quality or architecture.

2.3 Marketplace Success Principles

- Trust first: Every customer-facing feature should reduce buyer doubt.
- Seller quality over seller quantity: Poor sellers damage the marketplace brand.
- Profit per order matters more than vanity GMV.
- Admin control is not optional; marketplace operations require strong control panels.
- Automate later, control manually first: MVP should allow manual override with audit logs.
- Build small modules, test each one, then connect them.

3. User Roles and Permissions

Role	Main Permissions
Guest	Browse products, search, view product details, register or login.
Customer	Manage profile, cart, wishlist, orders, payments, returns, reviews, and support tickets.
Seller Owner	Manage seller profile, products, inventory, orders, returns, reports, payouts, and seller staff.
Seller Staff	Limited seller dashboard access based on assigned permissions.
Support Agent	View customers/orders/tickets and respond to support issues.
Operations Manager	Manage order flow, delivery issues, returns, seller performance, and escalations.
Finance Manager	Manage commissions, refunds, settlement batches, payment reconciliation, and payout reports.
Marketing Manager	Manage banners, coupons, campaigns, content, and promoted listing slots.
Admin	Manage major marketplace operations with configured permissions.
Super Admin	Full access to all modules, settings, financial rules, roles, permissions, and audit controls.

3.1 Strict Permission Rules

- A customer must never access seller or admin modules.
- A seller must never access another seller's orders, products, reports, customers, or payouts.
- Seller staff must only access modules assigned by the seller owner.
- Admin permissions must be role-based, not hardcoded only by user type.
- Finance users must not change products unless explicitly granted permission.
- Support users must not change commission, settlement, or payment settings.
- Every sensitive action must create an audit log entry.

4. System Modules

Module ID	Module Name
M01	Authentication and account management
M02	Customer profile and address management
M03	Seller onboarding, KYC, and seller management
M04	Product catalog, category, attribute, and variant management
M05	Search, filtering, sorting, and product discovery
M06	Cart, wishlist, and checkout
M07	Payment management: COD and digital payments
M08	Order management and multi-seller order splitting
M09	Inventory management
M10	Delivery and courier/logistics management
M11	Cancellation, return, exchange, and refund management
M12	Reviews, ratings, moderation, and seller scoring
M13	Coupons, campaigns, promotions, and promoted listings
M14	Commission, seller ledger, settlement, and finance
M15	Notifications: email, SMS, in-app, optional WhatsApp/push
M16	Customer support, dispute, and ticket management
M17	CMS: static pages, homepage banners, policies, buying guides
M18	Analytics, reports, dashboards, and exports
M19	Fraud, risk control, and trust signals
M20	Security, audit logs, settings, and system configuration

5. Functional Requirements

The following requirement tables define strict system behavior. Priority values mean: P0 = required for MVP, P1 = required soon after MVP, P2 = scale-stage or optional enhancement.

5.1 Authentication and Account Requirements

ID	Requirement	Priority	Acceptance Criteria
AUTH-001	The system shall allow customers to register using mobile number, email, or approved social login.	P0	Customer account is created only after required fields and verification rules are satisfied.
AUTH-002	The system shall support OTP or email verification for customer accounts.	P0	Unverified users cannot perform restricted actions when verification is enabled.
AUTH-003	The system shall allow login using email/password and mobile/OTP.	P0	Valid users can log in; invalid attempts show safe error messages.
AUTH-004	The system shall provide password reset and account recovery.	P0	Password reset token or OTP expires and cannot be reused.
AUTH-005	The admin panel shall have a separate secure login route.	P0	Customer/seller login cannot access admin routes.
AUTH-006	Admin accounts with sensitive permissions shall support two-factor authentication.	P1	Super admin can enforce 2FA for admin roles.
AUTH-007	The system shall rate-limit login and OTP requests.	P0	Repeated attempts are blocked or delayed based on policy.

5.2 Customer Storefront Requirements

ID	Requirement	Priority	Acceptance Criteria
CUST-001	The homepage shall show categories, search, banners, featured products, best sellers, trust badges, and policy links.	P0	Customer can navigate to category, search, cart, account, and policy pages.
CUST-002	Customers shall browse products by category, subcategory, brand, seller, price, rating, discount, and availability.	P0	Filters return accurate product sets and preserve active filter state.
CUST-003	The product detail page shall show title, images, price, stock, seller, seller verification, delivery estimate, return policy, variants, and reviews.	P0	Customer sees seller, final price, stock, and return eligibility before checkout.
CUST-004	Customers shall add, remove, and	P0	Cart recalculates subtotal, delivery,

	update products in cart.		discount, and final amount correctly.
CUST-005	The system shall validate stock, product status, seller status, price, and coupon before checkout.	P0	Invalid checkout is blocked with clear message.
CUST-006	Checkout shall show full payable amount before order confirmation.	P0	Subtotal, discount, delivery fee, COD fee, tax, and final amount are visible.
CUST-007	Customers shall view order history, order details, payment status, shipment status, and invoice if available.	P0	Order timeline is visible and updates correctly.
CUST-008	Customers shall request cancellation, return, exchange, or refund only when allowed by policy.	P0	Unavailable actions are hidden or blocked with reason.
CUST-009	Only verified buyers shall submit product reviews unless admin changes policy.	P0	Review submission is allowed only for delivered eligible order items.

5.3 Seller Dashboard Requirements

ID	Requirement	Priority	Acceptance Criteria
SELL-001	Sellers shall apply with business, owner, contact, payout, return address, and verification information.	P0	Seller application is stored as pending until admin decision.
SELL-002	Sellers shall not publish products until admin approval.	P0	Unapproved seller products never appear publicly.
SELL-003	Seller dashboard shall show orders, sales summary, pending actions, payout summary, low-stock alerts, and performance score.	P0	Dashboard displays only the seller's own data.
SELL-004	Sellers shall create and edit products with images, SKU, price, stock, category, description, attributes, variants, warranty, and return rules.	P0	Product cannot be submitted without required fields.
SELL-005	Seller-created products shall require admin approval unless trusted auto-approval is enabled.	P0	Pending products are not publicly visible.
SELL-006	Sellers shall manage inventory, stock status, and low-stock thresholds.	P0	Out-of-stock items cannot be purchased.
SELL-007	Sellers shall process assigned orders: confirm, pack, mark ready to ship, and respond to return requests.	P0	Seller cannot process orders belonging to another seller.
SELL-008	Sellers shall view payout ledger, commission deductions, refund deductions, and settlement history.	P0	Financial numbers match admin settlement records.
SELL-009	Seller owner shall manage seller staff and permissions.	P1	Staff access is limited to granted permissions.

5.4 Admin Panel Requirements

ID	Requirement	Priority	Acceptance Criteria
ADM-001	Admin dashboard shall show GMV, net revenue, orders, returns, active sellers, pending approvals, support tickets, and settlement status.	P0	Dashboard metrics match database reports for selected date range.
ADM-002	Admin shall approve, reject, request info, suspend, or reactivate sellers.	P0	Seller status changes are logged and reflected across platform access.
ADM-003	Admin shall approve, reject, edit, hide, suspend, or archive products.	P0	Public catalog only shows approved, published, in-stock, active-seller products.
ADM-004	Admin shall manage category hierarchy, attributes, return rules, and category commission.	P0	Product forms and commission calculations use the active category rules.
ADM-005	Admin shall manage all orders, statuses, delivery assignment, stuck orders, and order timeline.	P0	Order status changes are logged with actor and timestamp.

ADM-006	Finance admin shall configure commission rules globally, by category, by seller, or by product where enabled.	P0	Commission snapshot is stored at order placement.
ADM-007	Finance admin shall generate seller settlement batches and mark payouts as paid with proof or reference.	P0	Only eligible delivered orders outside hold period enter settlement.
ADM-008	Admin shall manage refund approvals, refund method, refund status, and settlement adjustment.	P0	Refund cannot process without linked order/payment item.
ADM-009	Admin shall manage users, roles, permissions, settings, notification templates, and audit logs.	P0	Only super admin can change critical system settings.

5.5 Product Catalog Requirements

ID	Requirement	Priority	Acceptance Criteria
PROD-001	Every product shall belong to one seller and one category.	P0	System blocks product save without seller and category references.
PROD-002	Each product shall support multiple images and one main image.	P0	Product cannot be published without at least one image.
PROD-003	Products shall support variants such as size, color, model, material, pack size, or other category-specific attributes.	P0	Each variant can have SKU, price, stock, and optional image.
PROD-004	Product statuses shall include draft, pending approval, approved, published, rejected, hidden, out of stock, suspended, and archived.	P0	Only published and approved products are visible to customers.
PROD-005	The system shall store price and product data snapshot inside order items.	P0	Past order details do not change when product price changes later.
PROD-006	Admin shall be able to define category-specific attributes.	P0	Product form changes based on selected category attributes.

5.6 Search and Discovery Requirements

ID	Requirement	Priority	Acceptance Criteria
SRCH-001	The system shall support keyword search across product name, category, brand, seller name, tags, and optional SKU.	P0	Relevant published products appear in search results.
SRCH-002	Search results shall support filters and sorting.	P0	User can sort by relevance, price, newest, best-selling, rating, and discount.
SRCH-003	The system should support typo tolerance and synonyms after MVP.	P1	Common misspellings still return relevant results.
SRCH-004	The system shall not return hidden, suspended, rejected, unapproved, or inactive-seller products.	P0	Restricted products never appear in search results.

5.7 Order Management Requirements

ID	Requirement	Priority	Acceptance Criteria
ORD-001	The system shall create a unique parent order for each customer checkout.	P0	Every placed order has a unique order ID.
ORD-002	If products come from multiple sellers, the system shall create seller-wise sub-orders.	P0	Each seller sees only their own sub-order.
ORD-003	Order statuses shall include pending, confirmed, processing, ready to ship, shipped, out for delivery, delivered, cancelled, return requested, returned, refund pending, refunded, failed delivery, and RTO.	P0	Order timeline reflects each status transition.
ORD-004	Inventory shall be reserved or reduced based on configured order	P0	Customer cannot buy more than available stock.

	policy.		
ORD-005	Order timeline shall record timestamp, actor, status, and remarks.	P0	Admin can audit every major order event.
ORD-006	Order cancellation shall be allowed only before configured status.	P0	Cancellation after shipment is blocked unless admin overrides.

5.8 Payment Requirements

ID	Requirement	Priority	Acceptance Criteria
PAY-001	The system shall support COD for MVP unless business policy disables it.	P0	COD orders are marked COD pending until delivery collection.
PAY-002	The system shall support digital payment gateway integration for mobile wallet/card payment.	P1	Gateway callback updates payment status securely.
PAY-003	Payment statuses shall include unpaid, pending, paid, failed, partially refunded, refunded, and cancelled.	P0	Payment state matches order and refund state.
PAY-004	The system shall never store raw card number, CVV, or sensitive card data.	P0	No sensitive card data exists in application database or logs.
PAY-005	Duplicate payment callbacks shall not duplicate paid status or revenue records.	P0	Idempotency prevents duplicate payment confirmation.
PAY-006	Payment reconciliation shall be possible using gateway transaction ID and order ID.	P1	Finance can match gateway report to platform records.

5.9 Delivery, Return, Exchange, and Refund Requirements

ID	Requirement	Priority	Acceptance Criteria
LOG-001	The system shall support platform-managed delivery and optional seller-managed delivery.	P0	Delivery method is stored per order/sub-order.
LOG-002	Delivery fee shall be calculated by flat rate, location, weight, order value, campaign, or free-delivery threshold.	P0	Checkout fee matches configured rules.
LOG-003	Courier integration should support shipment creation, label printing, tracking, COD collection, and return pickup.	P1	Courier status updates order shipment timeline.
RET-001	Customers shall request returns only for eligible products within the return window.	P0	Non-returnable or expired requests are blocked unless admin overrides.
RET-002	Return request shall include reason, description, item, and optional proof image/video.	P0	Admin/seller can review evidence before decision.
RET-003	Refund shall be processed only after approval and product verification unless policy allows instant refund.	P0	Refund status and settlement adjustments are linked.
RET-004	Exchange should be supported for eligible issues such as size, color, damaged product, or wrong product.	P1	Exchange creates replacement order or exchange workflow record.

5.10 Marketing, Coupons, Reviews, and Support Requirements

ID	Requirement	Priority	Acceptance Criteria
MKT-001	Admin shall create fixed amount, percentage, free delivery, category, seller, product, first-order, and minimum-order coupons.	P0	Coupon applies only when conditions are met.
MKT-002	Coupons shall support date range, total limit, per-user limit, minimum value, maximum discount, and restrictions.	P0	Coupon cannot exceed configured limits.
MKT-003	Admin shall create campaigns with banners, product lists, discount	P1	Campaign appears only within start/end date.

	rules, sellers, and landing pages.		
MKT-004	The system should support seller-paid promoted listings.	P1	Admin can configure slots, pricing, duration, and performance report.
REV-001	Verified customers shall submit product reviews with rating, text, and optional images.	P0	Review is linked to delivered order item.
REV-002	Admin shall moderate abusive, fake, or policy-violating reviews.	P0	Hidden reviews are not publicly visible.
SUP-001	Customers and sellers shall create support tickets with category, priority, description, attachments, and linked order if needed.	P0	Ticket can be assigned, updated, escalated, resolved, and closed.

5.11 Commission, Settlement, and Finance Requirements

ID	Requirement	Priority	Acceptance Criteria
FIN-001	The system shall calculate commission per order item using the applicable commission rule at order placement.	P0	Commission snapshot does not change later if commission settings change.
FIN-002	Seller ledger shall track sale amount, commission, delivery fee, refund deduction, penalty, adjustment, pending amount, and paid amount.	P0	Ledger total matches settlement report.
FIN-003	Settlement shall include only delivered orders outside the return-hold period.	P0	Undelivered or disputed items are excluded from payout.
FIN-004	Finance admin shall generate payout batches and export reports.	P0	Settlement batch contains seller, amount, order references, deductions, and payout status.
FIN-005	Financial records shall never be permanently deleted through normal admin UI.	P0	Records can be voided/adjusted but original audit trail remains.

6. Strict Marketplace Business Rules

These rules are mandatory. They protect trust, financial accuracy, seller accountability, and system reliability.

17. Do not allow unapproved sellers to publish products.
18. Do not allow unapproved products to appear publicly.
19. Do not allow customers to buy out-of-stock products.
20. Do not allow sellers to view another seller's orders, reports, customers, products, or payouts.
21. Do not store raw card data, CVV, or sensitive payment credentials.
22. Do not process seller payout before delivery and return-hold clearance.
23. Do not allow refund without a valid order item and payment reference.
24. Do not allow sensitive admin actions without audit logs.
25. Do not show hidden, suspended, rejected, or inactive-seller products to customers.
26. Do not change historical order item price after order placement; use adjustment records only.
27. Do not let coupons apply beyond configured limits.
28. Do not allow duplicate payment confirmation for the same order.
29. Do not permanently delete financial records; archive or void with audit trail.
30. Do not expose customer or seller private data publicly.
31. Do not launch production without backup, recovery, monitoring, and secure environment variables.
32. Do not let an AI agent change unrelated files while implementing a specific requirement.

6.1 Definition of Done for Each Feature

- Requirement ID is implemented.
- Database migration/model is created or updated.
- API endpoint or backend service is implemented.
- Frontend UI is implemented and validated.
- Admin control exists if the feature affects operations.

- Permission check is implemented.
- Input validation is implemented.
- Audit log exists for sensitive changes.
- Happy path and failure path tests are completed.
- Manual QA checklist is passed.
- AI-generated code has been reviewed by the developer before merge.

7. AI Agent Build Guide for Solo Developer

This section is written specifically for you as a solo developer building with AI agent support. The goal is not to let AI randomly build the system. The goal is to use AI as a controlled coding assistant while you remain the architect, reviewer, and final decision-maker.

7.1 AI Development Philosophy

- You are the product owner, architect, reviewer, and release manager.
- AI is a coding assistant, not the authority for business rules.
- The SRS is the source of truth. AI must follow this document, not invent different behavior.
- Every AI task must be small, testable, and linked to requirement IDs.
- Never ask AI to build the whole marketplace in one prompt.
- Never merge AI output without reading the changed files.
- Always run tests, linting, migrations, and manual QA before accepting AI work.

7.2 Recommended Repository Documentation Structure

```
/docs
  SRS.pdf
  MASTER_SPEC.md
  AI_AGENT_RULES.md
  DATABASE_SCHEMA.md
  API_CONTRACT.md
  FEATURE_BACKLOG.md
  TEST_PLAN.md
  SECURITY_CHECKLIST.md
  DEPLOYMENT_CHECKLIST.md
  CHANGELOG.md
/src
/tests
/database or /migrations
/scripts
```

7.3 Files You Should Maintain for AI Context

File	Purpose
MASTER_SPEC.md	Short version of the system vision, modules, and strict rules for AI context.
AI_AGENT_RULES.md	Rules AI must follow while editing code.
DATABASE_SCHEMA.md	Current database tables, relationships, indexes, and migration notes.
API_CONTRACT.md	Endpoint list, request/response formats, auth rules, and error codes.
FEATURE_BACKLOG.md	Requirement IDs mapped to development tasks.
TEST_PLAN.md	Required unit, integration, security, and manual QA tests.
SECURITY_CHECKLIST.md	Auth, RBAC, payment, file upload, and admin security checklist.
CHANGELOG.md	Human-readable record of completed changes.

7.4 AI Agent Rules File Template

```
# AI_AGENT_RULES.md
You are helping build a niche multivendor ecommerce marketplace.
Follow the SRS strictly.

Rules:
1. Implement only the assigned requirement IDs.
2. Do not change unrelated files.
```

3. Do not remove existing validations or permissions.
4. Do not bypass seller approval, product approval, payment checks, or audit logs.
5. Write clear, maintainable code.
6. Add or update tests for every feature.
7. Explain all changed files after completion.
8. If business logic is unclear, stop and ask for clarification.
9. Never store raw card data or secrets.
10. Use environment variables for credentials.
11. Never hardcode admin IDs, seller IDs, or payment statuses.
12. Keep database migrations reversible where possible.

7.5 How to Give Tasks to AI

Use small tasks. A good AI task should include requirement ID, current files, expected behavior, acceptance criteria, and tests.

Prompt Template:

You are working on my multivendor ecommerce marketplace.
 Read /docs/MASTER_SPEC.md and /docs/AI_AGENT_RULES.md first.
 Implement requirement: [REQ-ID]
 Feature: [feature name]
 Scope:
 - [specific file/module]
 - [specific behavior]
 Acceptance criteria:
 - [criterion 1]
 - [criterion 2]
 Tests required:
 - [test 1]
 - [test 2]
 Restrictions:
 - Do not change unrelated modules.
 - Do not modify existing business rules unless required.
 After coding, summarize changed files and how to test.

7.6 Bad AI Prompt vs Good AI Prompt

Bad Prompt	Good Prompt
Build the full seller dashboard.	Implement SELL-004 product creation form only. Include validation for name, category, SKU, price, stock, images, and status=pending_approval. Do not publish products until admin approval. Add tests.
Make checkout work.	Implement CUST-006 checkout summary calculation. Show subtotal, discount, delivery fee, COD fee, tax, and final payable amount. Add test cases for coupon and stock validation.
Add payments.	Implement PAY-002 payment initiation and callback handler using the selected gateway. Use idempotency and never store raw card data. Add tests for duplicate callback.
Fix bugs.	Investigate order status issue in ORD-003. Reproduce with test, explain root cause, patch only affected files, and add regression test.

7.7 AI-Assisted Build Workflow

33. Pick one requirement ID from the backlog.
34. Write or update the expected behavior in FEATURE_BACKLOG.md.
35. Ask AI to inspect relevant files and propose a plan before coding.
36. Review the plan. Remove anything that violates the SRS.
37. Ask AI to implement only that small feature.
38. Read the diff manually.
39. Run tests and linting.
40. Do manual browser/API testing.
41. Ask AI to generate missing edge-case tests.
42. Commit the change with requirement ID in commit message.
43. Update CHANGELOG.md and FEATURE_BACKLOG.md.

7.8 Recommended Git Workflow with AI

```
main
  - production-ready code only

develop
  - integration branch

feature/AUTH-001-customer-registration
feature/SELL-004-product-create
feature/ORD-002-multi-seller-order-split
feature/FIN-003-settlement-eligibility
```

- Use one feature branch per requirement or small group of related requirements.
- Use commit messages such as: feat(SELL-004): add seller product creation validation.
- Do not allow AI to commit directly without your review.
- Keep commits small so rollback is easy.

7.9 AI Checkpoints by Development Stage

Stage	AI Can Help With	You Must Verify
Planning	Break SRS into tasks, create backlog, suggest architecture.	Business rules, priority, and scope.
Database	Draft schema, indexes, migrations, relationships.	Data correctness, financial records, constraints, rollback.
Backend	Write controllers/services/validation/tests.	Permissions, business logic, edge cases, security.
Frontend	Create forms, dashboards, tables, states.	UX clarity, pricing visibility, mobile behavior.
Testing	Generate unit/integration/e2e tests.	Real marketplace scenarios and negative cases.
Security	Suggest checks and hardening.	Secrets, RBAC, payment safety, file upload risks.
Deployment	Generate scripts and checklists.	Environment, backups, monitoring, payment credentials.

7.10 AI Prompt Templates for Key Tasks

Architecture Review Prompt

Review my planned architecture for this multivendor marketplace against the SRS.
 Focus on seller isolation, order splitting, commission calculation, return/refund workflows, audit logs, and scalability.
 List risks, missing modules, and recommended fixes before coding.

Database Schema Prompt

Design the database schema for requirement IDs [IDs].
 Include tables, fields, relationships, indexes, constraints, and migration order.
 Check against these rules: sellers cannot see other sellers, financial records are auditable, orders store price snapshots, commission snapshots are stored at purchase time.

API Contract Prompt

Create API contract for [module] based on requirement IDs [IDs].
 Include endpoint, method, auth role, request body, response body, validation rules, error responses, and permission checks.
 Do not implement code yet.

Code Implementation Prompt

Implement requirement [REQ-ID] only.
 Use existing project conventions.
 Do not change unrelated files.
 Add validation, permission checks, tests, and audit log if needed.
 After implementation, explain changed files and test commands.

Bug Fix Prompt

Bug: [describe bug]
 Expected behavior: [expected]
 Actual behavior: [actual]
 Relevant requirement ID: [REQ-ID]
 First identify likely root cause and files involved.
 Then create a minimal fix with regression test.
 Do not refactor unrelated code.

Security Review Prompt

Review this module for security issues.
 Check RBAC, seller data isolation, input validation, file upload risks, XSS, SQL injection, CSRF, rate limiting, payment safety, secrets, and audit logs.
 Return a checklist of issues and fixes.

7.11 AI Safety Rules for Marketplace Finance

- Do not ask AI to simplify settlement logic without checking financial accuracy.
- Do not let AI remove order item price snapshots.
- Do not let AI calculate commission only from current product price.
- Do not let AI mark paid orders without verified gateway callback or COD collection confirmation.
- Do not let AI delete payment, refund, or settlement records.
- Always test duplicate payment callbacks, cancelled orders, returned orders, partial refunds, and multi-seller settlements.

7.12 AI Agent Output Review Checklist

- Does the code implement only the assigned requirement?
- Are permissions correct for customer, seller, admin, and finance roles?
- Can a seller access another seller's data? If yes, reject the code.
- Are validation errors clear and safe?
- Are financial calculations deterministic and stored as snapshots?
- Are migrations safe and reversible where possible?
- Are tests included for success, failure, and permission cases?
- Are secrets or credentials hardcoded? If yes, reject the code.
- Does the UI show required trust and pricing information clearly?
- Does the change affect existing modules unexpectedly? If yes, review carefully.

8. Recommended Technical Architecture

The SRS is technology-agnostic, but a solo developer should choose a stack that is productive, well-documented, and easy to deploy. Avoid over-engineering early. A modular monolith is usually better than microservices for MVP.

8.1 Recommended Architecture for Solo Developer

- Use a modular monolith for MVP with clear internal modules.
- Keep buyer storefront, seller dashboard, and admin dashboard separated by routes and permissions.
- Use one reliable relational database for core transactions.
- Use object storage for product images, verification documents, and support attachments.
- Use queues for notifications, payment callbacks, image processing, and report generation.
- Use cache for categories, homepage blocks, product lists, and frequent reads.
- Prepare clean module boundaries so services can be separated later if needed.

8.2 Suggested Stack Options

Layer	Recommended Options
Frontend	Next.js/React, Vue/Nuxt, or Laravel Blade/Inertia for faster solo development.
Backend	Laravel, NestJS, Django, or Spring Boot. Choose what you can maintain fastest.

Database	PostgreSQL recommended; MySQL acceptable.
Search	Database search for MVP; OpenSearch/Elasticsearch later.
Cache	Redis.
Queue	Redis Queue, RabbitMQ, or cloud queue.
Storage	S3-compatible object storage.
Deployment	Docker or managed VPS/cloud with staging and production environments.
Monitoring	Error tracking, uptime monitoring, logs, metrics.

8.3 Module Boundary Rules

- Auth module owns users, roles, permissions, sessions, OTP, and password reset.
- Seller module owns seller profile, verification, staff, and seller status.
- Catalog module owns products, categories, attributes, variants, inventory, and product approval.
- Order module owns cart checkout, parent orders, sub-orders, order items, and status timeline.
- Payment module owns payment initiation, callback, transaction records, COD collection, refunds, and reconciliation.
- Finance module owns commission, seller ledger, payout eligibility, and settlement batches.
- Support module owns tickets, disputes, attachments, and agent assignment.
- Admin module should orchestrate operations but not bypass core business rules.

9. Data Model and Main Entities

9.1 Core Entities

Entity	Purpose
users	All login identities: customers, sellers, admins, staff.
roles, permissions	RBAC definitions.
customer_profiles	Customer-specific data and preferences.
addresses	Customer and seller addresses.
seller_profiles	Seller business profile, status, payout info, return address.
seller_documents	KYC/business verification documents.
categories	Category and subcategory hierarchy.
attributes	Category-specific fields.
products	Main product record.
product_variants	SKU, variant attributes, stock, price.
inventory_logs	Stock movements and adjustments.
carts, cart_items	Customer cart records.
wishlists	Saved customer products.
orders	Parent customer order.
sub_orders	Seller-wise order split.
order_items	Purchased items with price snapshot.
order_timelines	Status events and audit flow.
payments	Payment attempts and statuses.
refunds	Refund requests and processing records.
shipments	Delivery/courier tracking.
returns	Return/exchange requests and decisions.
reviews	Product/seller reviews.
coupons, campaigns	Promotion tools.
commissions	Applied commission snapshots.
seller_ledgers	Seller financial ledger entries.
settlements	Payout batches and statuses.
support_tickets	Customer/seller support records.
notifications	Email/SMS/in-app notification records.
audit_logs	Critical system actions.

9.2 Essential Data Rules

- Product must belong to one seller and one category.
- Order item must store product name, SKU, seller ID, category ID, price, discount, tax, and commission snapshot at purchase time.
- Seller payout must be generated from ledger entries, not from recalculating old orders loosely.
- Payment must be linked to order and gateway transaction reference where applicable.

- Refund must be linked to order item, payment, and return request where applicable.
- Audit log must be append-only for normal application users.
- Verification documents must be private and access-controlled.
- Soft delete or archive should be used for financial records and historical order records.

10. API Requirements

10.1 API Design Rules

- All protected APIs must require authentication.
- Seller APIs must filter by authenticated seller ID, not seller ID from user input only.
- Admin APIs must check specific permission, not only admin role.
- API errors must be clear but not leak sensitive information.
- Payment callback APIs must verify signature or gateway authenticity.
- State-changing APIs must validate status transitions.
- All financial APIs must use server-side calculation, not client-submitted totals.
- Pagination is required for product, order, seller, customer, and report lists.

10.2 Required API Groups

API Group	Purpose
Auth API	Register, login, OTP, password reset, logout, session/user profile.
Customer API	Profile, addresses, wishlist, orders, returns, reviews, tickets.
Seller API	Seller profile, products, inventory, orders, returns, reports, payouts.
Catalog API	Categories, products, filters, search, product detail.
Cart/Checkout API	Cart items, coupon validation, checkout summary, order placement.
Payment API	Payment initiation, callback, status, refund request.
Order API	Order list, order details, timeline, status transitions.
Admin API	Seller approval, product moderation, orders, finance, settings, CMS, reports.
Notification API	Templates, logs, in-app notifications.
Support API	Tickets, messages, attachments, dispute workflow.

11. Non-Functional Requirements

Category	Requirement
Performance	Homepage under 3 seconds, product detail under 3 seconds, typical search under 2 seconds, checkout under 5 seconds excluding gateway delay.
Scalability	Design for 100,000+ products, 10,000+ sellers, 1,000,000+ customers, and campaign traffic spikes.
Availability	Target 99.5% uptime for MVP and 99.9% for growth stage.
Security	HTTPS, password hashing, RBAC, input validation, CSRF/XSS/SQL injection protection, rate limiting, audit logs.
Privacy	Protect customer data, seller payout data, and seller verification documents.
Reliability	Prevent duplicate order placement, duplicate payment confirmation, and invalid status transitions.
Maintainability	Modular code, documented APIs, migrations, tests, CI/CD, clear naming.
Usability	Mobile-first buyer UI and simple seller dashboard for non-technical users.
Localization	Support local currency, phone number, address format, and future Bangla/English UI.
Backup	Daily database backup, file backup, restore process, and tested disaster recovery.

12. Security and Fraud Requirements

12.1 Security Requirements

- Use HTTPS for all production traffic.
- Hash passwords with a modern secure algorithm.
- Expire OTPs and password reset tokens.
- Rate-limit login, OTP, password reset, and payment attempts.
- Validate and sanitize all user inputs.
- Protect file uploads with type, size, extension, and malware checks where possible.
- Store credentials and secrets only in environment variables or secret manager.
- Use secure cookies and CSRF protection where applicable.
- Add audit logs for seller approval, product approval, order status changes, payment changes, refunds, settlements, role changes, and settings changes.

12.2 Fraud and Risk Controls

- Track repeated COD failures by customer account, phone, and address pattern where allowed.
- Flag sellers with high cancellation, return, complaint, or fake product reports.
- Flag products with abnormal review patterns or high refund rate.
- Hold seller payouts during active disputes or return window.
- Require admin approval for suspicious refunds, large adjustments, or seller reactivation.
- Detect duplicate payment callbacks and repeated refund attempts.
- Allow admin to block abusive customers, sellers, products, and reviews.

13. Testing and Quality Assurance

13.1 Required Test Types

Test Type	Required Coverage
Unit tests	Business logic: commission, cart totals, coupon validation, status transitions, payout eligibility.
Integration tests	Order placement, payment callback, seller order visibility, return/refund workflow, settlement batch.
Permission tests	Customer/seller/admin boundaries and seller data isolation.
Security tests	Auth, RBAC, file upload, CSRF, XSS, SQL injection, rate limits.
E2E tests	Customer places order, seller processes order, admin settles payout.
Manual QA	Mobile UI, checkout clarity, admin operations, seller onboarding.
Performance tests	Search, product page, checkout, admin reports, campaign traffic.

13.2 Critical Test Scenarios

- Unapproved seller tries to create or publish product.
- Unapproved product appears in search - must not happen.
- Seller tries to open another seller's order by ID.
- Customer applies expired coupon or exceeds coupon limit.
- Customer places multi-seller order and each seller receives only their sub-order.
- Payment gateway sends duplicate callback.
- COD order is delivered and payment status updates correctly.
- Returned order is excluded from settlement.
- Refunded order adjusts seller ledger.
- Admin changes commission rate after an order - old order commission remains unchanged.
- Customer requests return after return window expires.
- Suspended seller products disappear from public catalog.

13.3 AI-Generated Test Prompt

Generate tests for requirement [REQ-ID].

Include:

- success case
- validation failure case
- permission failure case
- edge case
- regression case if relevant

Use existing project test style.

Do not change production code unless a missing testability issue is found and explained.

14. Deployment and DevOps Requirements

14.1 Environments

- Development: local developer environment.
- Staging: production-like environment for testing payment sandbox, courier sandbox, and QA.
- Production: live customer/seller system with secure credentials and monitoring.

14.2 Production Requirements

- SSL certificate and HTTPS enforced.
- Production environment variables configured securely.
- Database backup enabled and tested.
- File storage backup or lifecycle policy configured.
- Error monitoring enabled.
- Uptime monitoring enabled.
- Log rotation and log privacy configured.
- Payment gateway production credentials protected.
- Admin 2FA enabled for super admin and finance roles.
- Maintenance mode available.
- Rollback plan documented.

15. MVP Roadmap and Step-by-Step Build Plan

This roadmap is optimized for a solo developer using AI assistance. Do not start with everything at once. Build the system in modules, verify each module, and only then connect complex workflows.

15.1 Phase 0 - Project Foundation

- Create repository and documentation structure.
- Add SRS, AI_AGENT_RULES, MASTER_SPEC, DATABASE_SCHEMA, API_CONTRACT, TEST_PLAN.
- Choose tech stack and set up coding standards.
- Set up local development, database, migrations, seeders, linting, and testing framework.
- Create RBAC foundation and environment configuration.

15.2 Phase 1 - Core Identity and Admin Base

- Implement users, roles, permissions, login, registration, OTP/email verification.
- Create admin login and admin dashboard shell.
- Create audit log system.
- Create customer profile and address management.
- Test RBAC and unauthorized access thoroughly.

15.3 Phase 2 - Seller and Product Foundation

- Implement seller application, verification, approval, rejection, suspension.
- Implement category and attribute management.

- Implement seller product creation, product approval, product status, image upload.
- Implement variants, inventory, stock validation, and product moderation.
- Implement seller storefront and public product detail page.

15.4 Phase 3 - Buyer Shopping Flow

- Implement homepage, category page, search, filters, and product discovery.
- Implement wishlist and cart.
- Implement checkout summary with full price breakdown.
- Implement COD order placement.
- Implement order confirmation, order history, and order detail page.

15.5 Phase 4 - Order Operations

- Implement multi-seller parent order and sub-order logic.
- Implement seller order processing: confirm, pack, ready to ship.
- Implement admin order management and order timeline.
- Implement delivery fee rules and manual shipment tracking.
- Implement notifications for order events.

15.6 Phase 5 - Payment, Finance, and Settlement

- Implement digital payment gateway if selected.
- Implement payment callback verification and idempotency.
- Implement commission calculation with snapshots.
- Implement seller ledger.
- Implement settlement eligibility and payout batch generation.
- Implement finance reports and exports.

15.7 Phase 6 - Returns, Refunds, Reviews, and Support

- Implement cancellation rules.
- Implement return request, approval, pickup/received status, refund processing.
- Implement exchange workflow if needed.
- Implement verified buyer reviews and moderation.
- Implement support ticket and dispute management.

15.8 Phase 7 - Growth Features

- Implement coupons, campaign pages, and banners.
- Implement promoted listings and seller-paid visibility.
- Implement advanced analytics and seller performance score.
- Implement courier integration.
- Implement loyalty program or wallet/store credit.
- Prepare mobile app only after web platform is stable.

15.9 MVP Feature Priority

Must-Have MVP	Phase 2	Phase 3 / Scale
Auth, customer profile, seller application, admin approval	Digital payment gateway	Native mobile apps
Category/product management and product approval	Courier API integration	AI recommendations
Search, product detail, cart, checkout, COD	Promoted listings	Fulfillment service
Order management and seller sub-orders	Seller staff accounts	Seller financing partnership
Basic delivery tracking, returns, refunds	Advanced analytics	Live commerce
Commission and settlement reports	Loyalty/wallet	Private label operations

16. Final Launch Checklist

16.1 Product and Business Readiness

- Niche and first categories finalized.
- Seller onboarding process tested.
- At least initial verified sellers added.
- Product approval workflow tested.
- Return, refund, delivery, privacy, terms, and seller policies published.
- Customer support process ready.
- Commission and settlement policy finalized.
- Delivery partner and COD handling process confirmed.

16.2 Technical Readiness

- Production deployment completed.
- HTTPS active.
- Backups tested.
- Payment/courier credentials secured.
- Admin 2FA enabled.
- Error monitoring active.
- Uptime monitoring active.
- Logs configured.
- Security checklist passed.
- Performance checked on mobile and desktop.
- Rollback plan ready.

16.3 Marketplace Readiness

- Seller cannot access other seller data.
- Unapproved products do not appear publicly.
- Out-of-stock products cannot be purchased.
- Checkout shows final amount clearly.
- Multi-seller order splitting works.
- Commission calculation is correct.
- Settlement excludes active disputes and return-hold orders.
- Refund adjusts seller ledger correctly.
- All critical admin actions create audit logs.
- Customer can complete a full order journey from browse to delivery.

17. Final Recommendation

Build the platform in controlled stages. The most dangerous mistake is to build only a product catalog and checkout while ignoring seller verification, product approval, order splitting, finance, returns, refunds, permissions, and audit trails.

As a solo developer using AI agents, your strongest advantage is speed. Your biggest risk is uncontrolled complexity. Use this SRS as the source of truth, divide the system into small requirement-based tasks, ask AI to implement one task at a time, review every diff, run tests, and keep marketplace rules strict from day one.

The first production version should be simple but correct. It should prove real order flow, seller quality, customer trust, payment safety, and settlement accuracy. After that, scale with campaigns, promoted listings, courier automation, seller subscriptions, loyalty, and mobile apps.